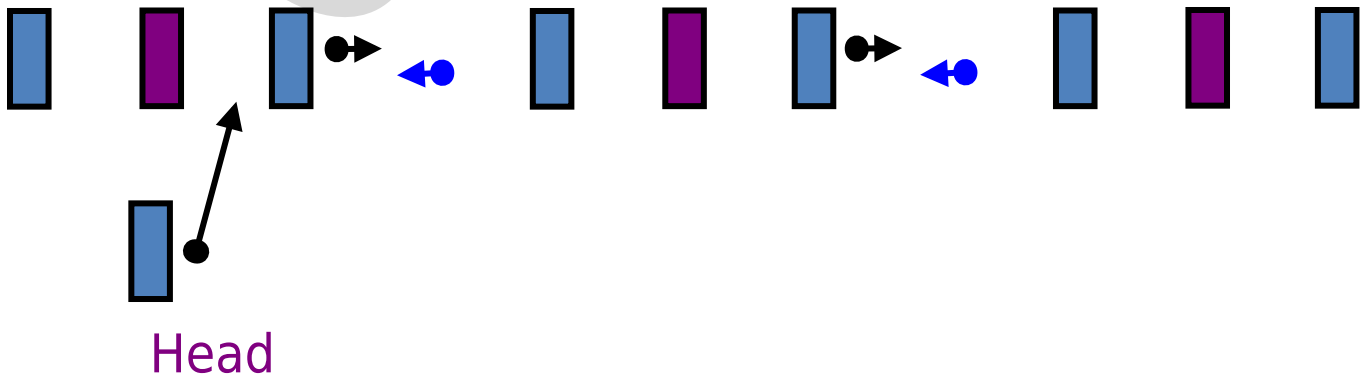
Lecture 8
Variations of Linked List

Prepared By:

Afsana Begum
Senior Lecturer,
Software Engineering Department,
Daffodil International University.

Variations of Linked Lists

- ***Doubly linked lists***
- *A linked list in which each node has three parts :one information and 2 pointers:*
- *An information field which contains the data of node.*
- *a forward pointer (a pointer to the next node in the list) and*
- *a backward pointer (a pointer to the node preceding the current node in the list) is called a doubly linked list. Here is a picture:*



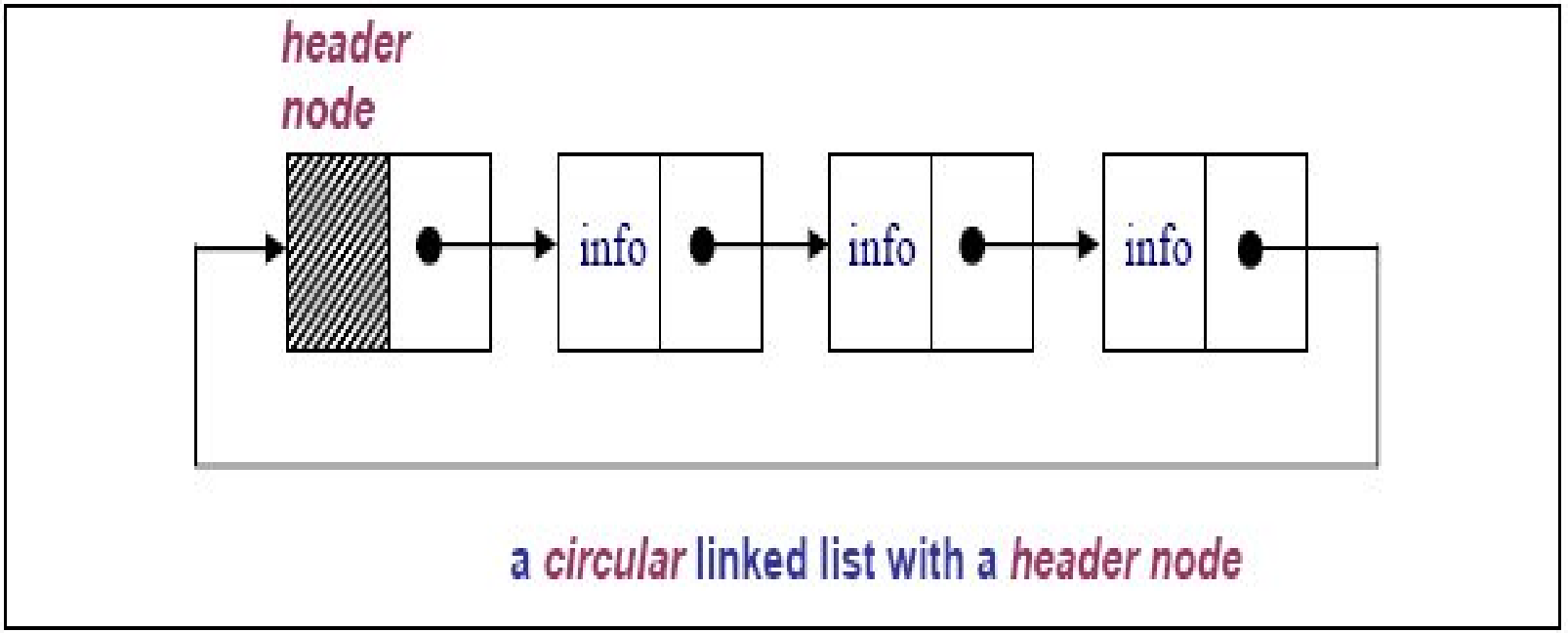
Each node points to not only successor but the predecessor
There are two NULL: at the first and last nodes in the list

–Advantage: given a node, it is easy to visit its predecessor. Convenient to traverse lists backwards.

The primary **disadvantage** of doubly linked lists are that
(1) Each node requires an extra pointer, requiring more space, and
(2) The insertion or deletion of a node takes a bit longer (more pointer operations).

Variations of Linked Lists

- *Circular linked lists*
  - The last node points to the first node of the list
  - How do we know when we have finished traversing the list? (Tip: check if the pointer of the current node is equal to the head.)

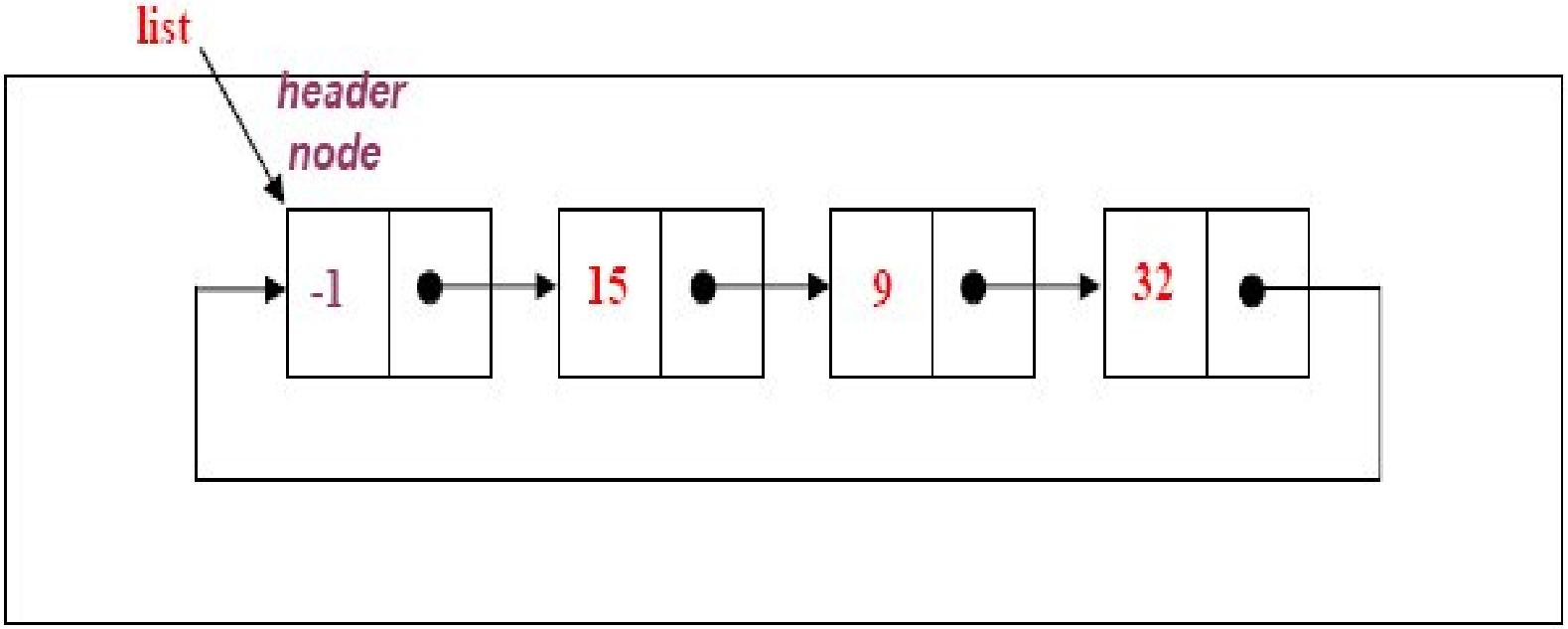


Variations of Linked Lists

circular linked list:

- In a circular linked list there are two methods to know whether a node is the first node or not.
  - Either a external pointer, ***list***, points the first node or
  - A ***header node*** is placed as the first node of the circular list.
- The header node can be separated from the others by either heaving a ***sentinel value*** as the info part or
- having a dedicated ***flag*** variable to specify if the node is a header node or not.

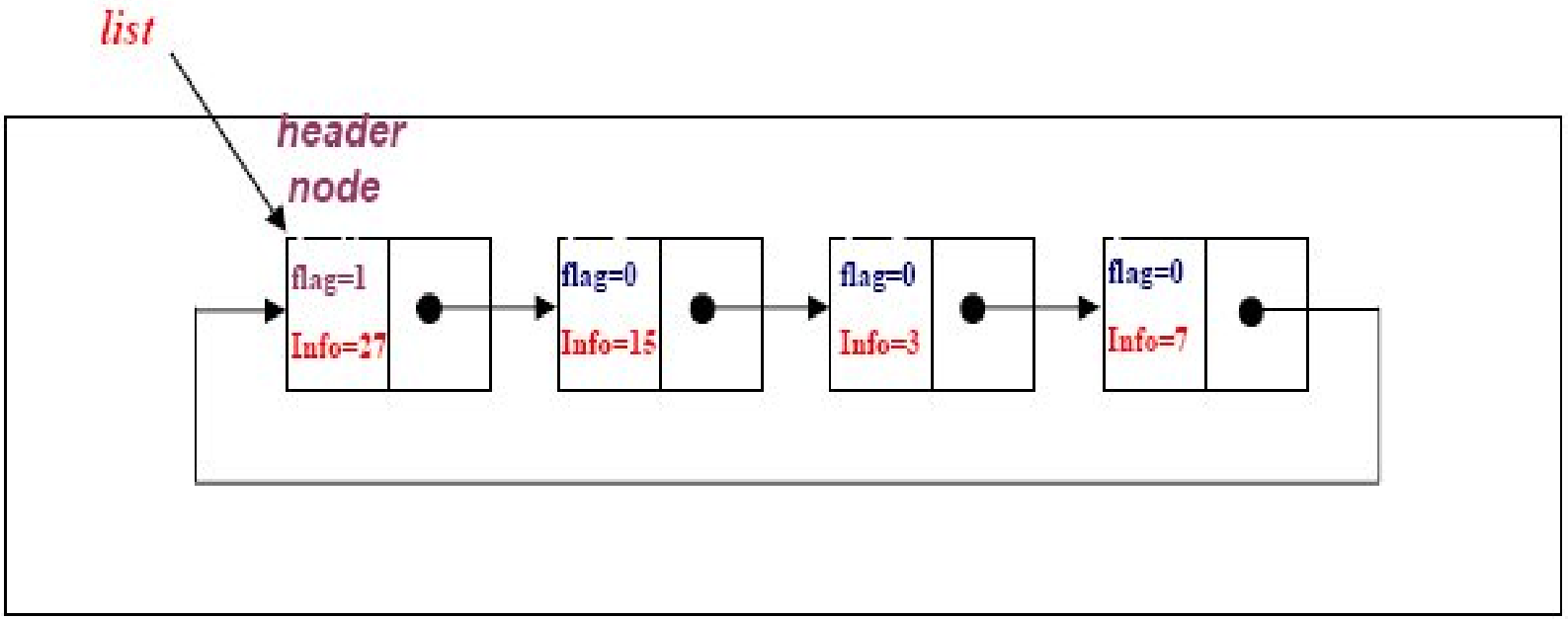
CIRCULAR LIST with header node (With ***Sentinel value***)



•The header node in a circular list can be specified by a ***sentinel value*** or a dedicated ***flag***:

•***Header Node with Sentinel:*** Assume that info part contains positive integers. Therefore the info part of a header node can be -1.

CIRCULAR LIST with header node (With Flag)



- ***Header Node with Flag:*** In this case a extra variable called flag can be used to represent the header node.
- For example flag in the header node can be 1, where the flag is 0 for the other nodes.

Example of application of circular linked list

- A good example of an application where circular linked list should be used is a timesharing problem solved by the operating system.
- In a timesharing environment, the operating system must maintain a list of present users and must alternately allow each user to use a small slice of CPU time, one user at a time.
- The operating system will pick a user, let him/her use a small amount of CPU time and then move on to the next user, etc.
- For this application, there should be no NIL pointers unless there is absolutely no one requesting CPU time.

Advantages

- Each node is accessible from any node.
- **Disadvantage:**
- Danger of an infinite loop !

Linear linked list vs Circular Linked Lists

- In **linear linked** lists if a list is traversed (all the elements visited) an external pointer to the list must be preserved in order to be able to reference the list again.
- **Circular linked** lists can be used to help the traverse the same list again and again if needed. A circular list is very similar to the linear list where in the circular list the pointer of the last node points not NULL but the first node.

Array versus Linked Lists

- **Linked lists** are more complex to code and manage than arrays, but they have some distinct advantages.

- **Dynamic:** a linked list can easily grow and shrink in size.
  - We don't need to know how many nodes will be in the list. They are created in memory as needed.
  - In contrast, the size of a C++ array is fixed at compilation time.

- **Easy and fast insertions and deletions**
  - To insert or delete an element in an array, we need to copy to temporary variables to make room for new elements or close the gap caused by deleted elements.
  - With a linked list, no need to move other nodes. Only need to reset some pointers.